

SAMPLE QUESTIONS:

Question 1 — AIOps Log Anomaly Detection

You are working as an AIOps engineer for an application server. The server generates logs containing CPU usage, memory usage and response time.

Create a Python program that:

1. Creates or reads a sample dataset containing:
 - Timestamp
 - CPU Usage
 - Memory Usage
 - Response Time
2. Calculates basic statistics for the metrics.
3. Detects anomalous values using a simple threshold-based approach.
4. Prints the anomalous records.
5. Displays a graph showing the metric values and anomalies.

Expected output:

Total records: 20

Anomalies detected: 3

Timestamp	CPU	Status
10:05	95%	ANOMALY
10:12	97%	ANOMALY
10:18	92%	ANOMALY

Concepts tested:

AIOps fundamentals, logs, metrics, anomaly detection.

Question 2 — Kafka Topic and Producer

Set up a Kafka environment and create a topic called:

server_metrics

Perform the following:

1. Start the Kafka server/cluster.

2. Create the `server_metrics` topic.
3. Configure a Kafka producer.
4. Send at least **10 server metric messages**.

Each message should contain:

server_id
cpu_usage
memory_usage

Example:

```
{  
  "server_id": "server01",  
  "cpu_usage": 82,  
  "memory_usage": 65  
}
```

5. Verify that the messages are successfully published to the topic.

Concepts tested:

Kafka cluster, topic, producer, Kafka practical.

Question 3 — Python Kafka Consumer

Write a Python Kafka consumer that consumes messages from:

server_metrics

The consumer should:

1. Connect to the Kafka broker.
2. Subscribe to the `server_metrics` topic.
3. Continuously receive messages.
4. Display the received server metrics.
5. Detect whether CPU usage is greater than **80%**.
6. Print:

ALERT: High CPU detected on server01

when the condition is satisfied.

Example:

Received:

Server: server01
CPU: 85%
Memory: 62%

ALERT: High CPU detected on server01

Concepts tested:

Consumer, topic, Python Kafka consumer, real-time monitoring, anomaly detection.

Question 4 — Build an AIOps Workflow using Airflow

Create an Apache Airflow DAG representing a **basic AIOps workflow**.

The workflow should contain the following tasks:

Collect Metrics
↓
Process Metrics
↓
Detect Anomaly
↓
Generate Report

Requirements

Task 1 — **collect_metrics**

Use **PythonOperator** to generate/sample server metrics.

Example:

CPU = 87
Memory = 65
Response Time = 420ms

Task 2 — **process_metrics**

Process the collected metrics and print them.

Task 3 — **detect_anomaly**

Check whether:

CPU > 80

If yes, print:

Anomaly detected: High CPU usage

Otherwise:

No anomaly detected

Task 4 — generate_report

Print a final AIOps report:

```
===== AIOps Report =====  
Metrics collected successfully  
Metrics processed successfully  
Anomaly detection completed  
=====
```

DAG Requirements

Your DAG must:

- Use `PythonOperator`.
- Define all four tasks.
- Define the correct dependencies.
- Run the tasks in the following order:

```
collect_metrics >> process_metrics >> detect_anomaly >> generate_report
```

Concepts tested:

Airflow, DAG, PythonOperator, dependencies, AIOps lifecycle, basic AIOps workflow.

Question 5 — Integrated AIOps Challenge

Modify your Kafka consumer from Question 3 so that it behaves like a simple AIOps monitoring system.

The consumer should:

1. Receive server metrics from Kafka.
2. Check CPU usage.
3. Detect an anomaly when CPU > 80%.
4. Print an alert.
5. Maintain a count of detected anomalies.

Example:

Message received: server01 | CPU: 85%
ALERT: High CPU detected

Message received: server02 | CPU: 45%
Normal

Message received: server03 | CPU: 91%
ALERT: High CPU detected

Total anomalies detected: 2

Concepts tested:

Kafka + Python + metrics + anomaly detection + AIOps monitoring.